

Q1)

Arrays in C do not perform bound checking, which makes programs run with minimum effort/overhead. This mainly affects which language evaluation measure?

- (a) Simplicity and readability
- (b) Reliability and orthogonality
- (c) Cost of execution and reliability
- (d) Writability and others

Correct answer: (c) Cost of execution and reliability

EBNF grammar

```
<start>    ::= <statement>
           | <statement> <start>
<statement> ::= <name> "=" <value> ";"
           | "print" "(" <value> ")"
<value>    ::= <name>
           | <number>
           | <text>
           | <statement> <statement>
<name>     ::= <letter> { <letter> | <digit> | "_" }
<number>   ::= <digit> { <digit> }
<text>     ::= "" { <character> } ""
```

Input example used with this grammar:

```
x = 5; y = 6; print("done");
```

Q2

How many distinct parse trees can theoretically be generated using this grammar for the given example input?

- (a) 1
- (b) 2
- (c) 3

☒ (d) more than 3

Correct: (d) more than 3

Q3) Which grammar rule creates the ambiguity in this grammar?

(a) $\langle \text{value} \rangle \rightarrow \langle \text{name} \rangle | \langle \text{number} \rangle | \langle \text{text} \rangle$

(b) $\langle \text{start} \rangle \rightarrow \langle \text{statement} \rangle | \langle \text{statement} \rangle \langle \text{start} \rangle$

☒ (c) $\langle \text{statement} \rangle \rightarrow \langle \text{statement} \rangle \langle \text{statement} \rangle$

(d) $\langle \text{name} \rangle \rightarrow \langle \text{letter} \rangle | \langle \text{letter} \rangle \langle \text{digit} \rangle | \text{" "}$

Correct: (c) $\langle \text{value} \rangle \rightarrow \langle \text{statement} \rangle \langle \text{statement} \rangle$

Q4) Which of the following statements best explains why the grammar is not suitable for top-down (LL) parsing?

- (a) It contains multiple token definitions that are identical.
- ☒ (b) It has left recursion and overlapping derivations.
- (c) It allows string concatenation in $\langle \text{text} \rangle$.
- (d) It has ambiguous terminal symbols for $\langle \text{value} \rangle$.

Correct: (b)

Q5) Which parsing technique would most easily reveal this grammar's ambiguity during implementation?

- ☒ (a) Recursive-descent parsing
- (b) Shift-reduce (LR) parsing
- (c) Predictive LL(1) parsing
- (d) Deterministic finite automaton (DFA)–based analysis

Correct: (a) Recursive-descent parsing

Q6) Which regular expression correctly describes an **even binary counter** (all binary strings for even numbers)?

- ☒ (a) $(0|1)^* 0$
- (b) $(0|1)^* 1$
- (c) $0 (0|1)^*$
- (d) $(00)^*$

Correct: (a) $(0|1)^*0$

Q7) Which of the following best describes the language generated by the regular expression

$(1^*01^*01^*)^*?$

(a) All combinations of zeros and ones.

(b) All strings with at least two zeros.

☒ (c) All strings with an even number of zeros.

(d) All strings that end with 01.

Correct: (c)

Q8) Is $(a^* | b^*)^*$ equivalent to $(a | b)^*$?

Options:

- ☒ (a) True
- (b) False

Correct: (b)

Q9) Which of the following best describes the regular expression: $[1-9] \text{ digit}^*$

- (a) All digit combinations
- (b) At most one zero
- ☒ (c) All digits with no leading zeros
- (d) At least one zero

Correct: (c)

Q10) Which of the following best describes the regular expression:

$((\epsilon | b | bb)a)^* (\epsilon | b | bb)$

- (a) All combinations of a's and b's
- (b) Even number of b's
- (c) Odd number of b's
- (d) No three consecutive b's

Correct: (d) No three consecutive b's

Q11) Which regular expression describes all strings over $\{a,b\}$ that contain two a's?

(a) $b^* a b^* a b^*$

(b) $ab^* ab^*$

(c) $b a b^* ab^*$

(d) $b^* ab^* ab$

Correct: (a)

Q12) Which of the following is a property of a deterministic finite automaton (DFA)?

- (a) It may have multiple transitions for the same input symbol from a state.
- (b) It may use ϵ -transitions.
- (c) For each state and each input symbol, there is **exactly one outgoing transition**.
- (d) It can recognize non-regular languages.

Correct: (c)

Q13) Compiler uses nondeterministic finite automata to do the necessary matching.

- (a) True
- (b) False

Correct: (b) False

Q14) What is the relationship between the languages recognized by DFAs and NFAs?

- (a) DFAs recognize a strict subset of the languages recognized by NFAs.
- (b) NFAs recognize a strict subset of the languages recognized by DFAs.
- (c) DFAs and NFAs recognize **exactly the same class of languages (regular languages)**.
- (d) Their languages are disjoint.

Correct: (c)